

Final Project Report

SkyLane: A Stateful Spatial Streaming Architecture for Next-Generation UTM/UAM

Student Saad Dastgir (CWID ID: 20370664)

Course CISC 560 - Big Data and Analytics

Instructor Dr. Agnideven Palanisamy Sundar

Term Spring 2026

Abstract

This report presents SkyLane, a Big Data system designed to support real-time Unmanned Aircraft System Traffic Management (UTM) and Urban Air Mobility (UAM) operations over a dense metropolitan airspace such as the New York region. SkyLane must ingest more than one million telemetry events per second from approximately 100,000 concurrent autonomous aircraft, generate collision-avoidance alerts at sub-50-millisecond end-to-end latency, and persist 15–20 TB of flight data per day for regulatory replay and machine learning training. The central architectural insight is that spatial proximity, not device identity, must drive partitioning. By keying ingestion and stream processing on hierarchical hexagonal cells (H3) rather than aircraft_id, the system collapses an otherwise intractable $O(n^2)$ collision-detection problem into an $O(n \cdot k)$ workload that parallelizes cleanly across a Flink cluster. The design combines a Kafka ingestion layer, a stateful Flink streaming engine, a tiered storage substrate (Redis → Apache Pinot → Delta Lakehouse on S3), and embedded ONNX-based machine learning inference.

The report quantifies scale, defends every architectural choice against an explicit tradeoff, predicts failure modes, and proposes recovery semantics consistent with aviation safety standards.

1. Introduction and Problem Definition

1.1 The Real-World Problem

The integration of autonomous drones and electric vertical takeoff and landing (eVTOL) air taxis into low-altitude urban airspace is no longer a speculative scenario. The Federal Aviation Administration's UTM Concept of Operations [1] and NASA's UAM vision [2] both anticipate hundreds of thousands of small unmanned aircraft operating below 400 feet over major metropolitan regions within the current decade. Companies including Joby, Wisk, Zipline, and Amazon Prime Air have already begun Beyond Visual Line of Sight (BVLOS) operations under FAA Part 135 waivers. This emerging airspace presents a fundamental safety problem: human visual navigation does not scale. A single air traffic controller can safely sequence approximately ten to twenty aircraft. Manhattan alone, at peak demand, may host tens of thousands of simultaneous drone deliveries, eVTOL passenger trips, and emergency-response sorties. The only path to safe operation is automated, deterministic, real-time spatial deconfliction backed by a distributed data infrastructure. Skylane addresses this problem. It is an airspace monitoring, routing, and collision-avoidance engine that ingests continuous telemetry from autonomous aircraft, maintains the immediate state of the entire airspace, and produces sub-second routing decisions and alerts.

1.2 Why This Problem Requires a Big Data Solution

Three independent forces eliminate any single-machine or traditional database solution:

Velocity. A telemetry rate of 10 Hz across 100,000 aircraft yields one million events per second. The fastest open-source single-node OLTP database, PostgreSQL on enterprise hardware, sustains approximately 50,000 writes per second [3].

The system is therefore ingestion-bound at twenty times the capacity of any single node.

State. Collision detection is not a stateless query. Predicting whether two aircraft will intersect requires their last several seconds of position, velocity, and acceleration vectors. The system must maintain a rolling window of state for every aircraft and update it ten times per second. This is canonical stateful streaming territory.

Geometry. The natural join key is spatial proximity, not a primary key. A naive cross-join of 100,000 aircraft produces 10^{10} pairwise distance computations every 100 milliseconds - physically impossible on any single machine. The system must partition the workload by geography and exploit spatial locality. These three forces - velocity, state, and geometry - are precisely the conditions under which Big Data architectures are mandatory rather than optional [4].

1.3 Quantified Scale

Dimension	Value
Concurrent active aircraft	100,000
Telemetry sample rate per aircraft	10 Hz
Total ingestion rate	1,000,000 events/sec
Average telemetry payload	~200 bytes
Average health-metric payload	~500 bytes (1 Hz)
Daily raw data volume	15–20 TB
Annualized data volume	~6 PB
Collision-alert end-to-end latency (p99)	< 50 ms
Operator dashboard refresh	< 1 sec
Regulatory retention (cold tier)	7 years
Queryable retention (warm tier)	30 days
Primary users	UAS Service Suppliers (USS), FAA / ICAO regulators, fleet operators, automated city dispatchers, public safety agencies

These numbers are not aspirational; they are derived directly from the FAA's UTM CONOPS v2.0 demand model [1] and consistent with industry projections from McKinsey's UAM market analysis [5].

2. System Requirements

2.1 Functional Requirements

1. **F1 - Ingest** continuous telemetry streams (position, velocity, attitude) and health metrics (battery, motor RPM, payload weight) from all active aircraft.
2. **F2 - Maintain a real-time spatial index** of every aircraft in the airspace.
3. **F3 - Detect imminent collisions** between aircraft within a configurable time-to-intersection horizon (default 3 seconds).
4. **F4 - Enforce dynamic geofences** including no-fly zones, temporary flight restrictions, and weather-driven exclusions.
5. **F5 - Predict trajectories** five seconds into the future using a learned model.
6. **F6 - Detect operational anomalies** such as inferred unauthorized payload, motor degradation, or lost-link events.
7. **F7 - Persist all events durably** for accident replay and regulatory audit.
8. **F8 - Serve dashboards and APIs** to fleet operators, regulators, and automated dispatchers.
9. **F9 - Train and deploy machine learning models** for trajectory prediction and anomaly detection.

2.2 Non-Functional Requirements

Category	Requirement
Latency	Collision alerts: p99 < 50 ms end-to-end. Dashboard refresh: < 1sec.
Throughput	Sustained 1 M events/sec with 5× headroom for surge events.
Availability	99.999 % ("five nines"); approximately 5 minutes of unplanned downtime per year.
Durability	Eleven-nines (10^{-11}) data loss probability for regulatory tier, consistent with S3 SLA [6].
Consistency	Exactly-once semantics on the safety-critical hot path; eventual consistency acceptable on analytics tier.
Scalability	Horizontal scaling from 20 TB/day to 200 TB/day without architectural redesign.
Cost	Operating cost target $\leq$ \$0.0001 per processed event (\$100 per million events).
Governance	FAA Part 91/107 audit trail; immutable raw layer; row- and column- level access control.
Security	TLS 1.3 in transit; AES-256 at rest; mutual TLS between aircraft and ingestion broker; aircraft IDs hashed in analytics tiers.
Privacy	Operator and passenger PII isolated from telemetry tables; column- level masking for non-privileged consumers.
Multi-region	Active-active across two AWS regions; zero RPO; RTO < 60 sec.

3. Architecture Design

3.1 Architecture Overview

Skylane is organized into five layers, each with a distinct latency profile and a clearly bounded responsibility. The layering is deliberately strict: the hot path that produces collision alerts never traverses the warehouse, and the analytics path never blocks streaming.

3.2 Component Roles and Justification

Layer 1 - Edge & Ingestion. Aircraft publish over MQTT, the IoT-standard publish/subscribe protocol, to regional edge brokers because MQTT is bandwidth-efficient over lossy 5G links and supports last-will testaments for lost-link detection [7]. A bridge translates MQTT into Kafka, the de-facto industrial backbone for high-throughput durable streaming [8]. Kafka is chosen over Kinesis or Pulsar for three reasons: (a) Kafka's tiered storage allows 24-hour replay without paying for SSD at petabyte scale; (b) Kafka transactional producers provide the foundation for exactly-once semantics downstream; (c) Kafka's open protocol prevents regulatory lock-in. The single most important decision in the entire architecture appears here: **the telemetry-hot topic is partitioned by H3 cell, not by aircraft_id**. This decision is justified in detail in Section 5.

Layer 2 - Stream Processing. Apache Flink is the processing engine [9]. Flink is selected over Spark Structured Streaming and Kafka Streams because it offers (a) true event-time processing with watermarks, (b) keyed state with incremental RocksDB checkpoints, (c) sub-50-millisecond tail latencies under sustained load, and (d) exactly-once end-to-end guarantees when paired with Kafka transactional sinks [10]. Spark Structured Streaming, by contrast, is micro-batch and cannot meet the 50 ms SLA. Three Flink jobs run concurrently: a spatial-join job that detects imminent collisions, a broadcast-state job that detects geofence breaches, and an inference job that runs ONNX models against streaming features.

Layer 3 - Storage. Three tiers are mandatory because no single engine satisfies all three latency profiles simultaneously. Redis serves microsecond key/value lookups for online features. Apache Pinot serves sub-second analytical queries over the last 30

days [11]. Delta Lake on S3 serves long-horizon batch queries and is the immutable system of record [12].

Layer 4 - Serving. Operators consume real-time alerts over WebSockets directly from Kafka's alerts topic, bypassing storage. Dashboards are powered by Pinot for sub-second refresh. Trino (formerly PrestoSQL) provides federated SQL across Delta, Pinot, and Redis for ad-hoc regulatory queries [13].

Layer 5 - Machine Learning. Spark trains models nightly against Delta Silver. Models are exported to ONNX and hot-swapped into Flink TaskManagers without restarting the pipeline. Feast manages the feature store with split offline (Delta) and online (Redis) backends [14].

3.3 Data Flow Walkthrough

A single drone telemetry event traverses the system as follows:

1. **t = 0 ms.** Drone publishes a 200-byte CBOR-encoded telemetry message to its regional MQTT broker over a 5G connection.
2. **t = 8 ms.** MQTT-Kafka bridge enriches the message with an H3 cell ID at resolution 9 (computed from lat/lon) and writes it to the telemetry-hot Kafka topic, partitioned by that H3 cell.
3. **t = 14 ms.** Flink Job A consumes the event. Because Flink's parallelism is keyed on the same H3 cell, the event arrives at the TaskManager that already holds state for every aircraft in that cell.
4. **t = 18 ms.** Job A updates the rolling 3-second state for the aircraft, computes pairwise time-to-intersection against every other aircraft in the 7-cell ring (target plus six neighbors), and runs the ONNX trajectory model.
5. **t = 32 ms.** If a collision is predicted within the horizon, an alert is emitted to the alerts Kafka topic (transactional). The enriched telemetry record is simultaneously written to the enriched topic, which is mirrored to Delta Bronze by a Kafka Connect S3 sink.

6. **t = 40 ms.** The alert arrives at the operator's UI via the WebSocket gateway.
7. **t = 50 ms.** Total p99 latency budget consumed.
8. **Asynchronously.** A Kafka Connect sink lands the raw event in Delta Bronze. Pinot's real-time table also consumes from Kafka and indexes the event for OLAP within ~1 second. Nightly, a Spark job reads Delta Silver and retrains the trajectory model.

4. Storage Design

4.1 The Three-Tier Lakehouse

Tier	Engine	Format	Partitioning	Retention	Schema Discipline
Hot	Redis Cluster + ScyllaDB	In-memory + LSM	Hash on aircraft_id	TTL 60 s	Schema-on-write (state vector struct)
Warm	Apache Pinot	Columnar (segment)	date, H3_cell_res6	30 days	Schema-on-write (typed columns)
Cold-Bronze	S3 + Delta Lake	Parquet + ZSTD	date / H3_cell_res6 / hour	7 years	Schema-on-read (raw)
Cold-Silver	S3 + Delta Lake	Parquet + ZSTD	date / H3_cell_res6	7 years	Schema-on-write (validated)
Cold-Gold	S3 + Delta Lake	Parquet + ZSTD	date / operator / H3_cell_res6	7 years	Schema-on-write (aggregated)

This is a canonical **lakehouse** [15] rather than either a pure data lake (no ACID, no schema enforcement) or a pure warehouse (too rigid, too expensive). The **medallion architecture** [16] — Bronze for raw, Silver for cleaned, Gold for aggregated — provides progressive refinement and clear governance boundaries.

4.2 Format Choice

Parquet is the storage format for all cold and warm data because it is columnar (essential for Pinot and Trino), strongly typed (essential for governance), supports predicate pushdown and statistics-based file pruning, and is the open standard underlying both Delta Lake and Apache Iceberg [17]. ZSTD is the compression codec because it offers a roughly 10–15 % size advantage over Snappy at decompression speeds adequate for analytical workloads [18]. For online state, Redis uses its native binary protocol; for ScyllaDB, a custom flatbuffer representation of the aircraft state vector (32 fields, ~256 bytes) avoids JSON parsing on the hot read path.

4.3 Partitioning Strategy

Partitioning is the single most consequential storage decision in any Big Data system. Skylane partitions cold tables on three dimensions in this order: **date** → **H3 cell at resolution 6** → **hour**.

- Date is the primary partition because almost every regulatory query is bounded by a date range.
- H3 res-6 is the secondary partition because spatial queries are the second most common access pattern (e.g., "all aircraft over Manhattan during incident X"). Resolution 6 yields hexagons of approximately 36 km² — coarse enough that a single partition is multi-megabyte (efficient for Spark scan), fine enough that a single neighborhood query prunes 99 %+ of partitions.
- Hour is the tertiary partition for sub-day temporal pruning.

Within each partition, files are **Z-ordered on H3_cell_res9 and aircraft_id** [19]. This co-locates spatially adjacent records within the same Parquet row group, dramatically improving compression and predicate pushdown.

4.4 Schema-on-Read vs Schema-on-Write

Bronze is schema-on-read because telemetry schemas evolve frequently as aircraft manufacturers add new fields (e.g., new sensor types, new health metrics). Forcing

schema-on-write at ingestion would create a brittle pipeline that drops or rejects valid future data. Silver and Gold are schema-on-write because downstream consumers — dashboards, ML pipelines, regulatory reports — depend on stable contracts. Silver enforces type checking, range validation (no negative altitudes, no GPS coordinates outside the operating region), and deduplication. Schema evolution is governed via Delta Lake's column-level evolution semantics.

4.5 Governance and Retention

Governance is enforced via **Unity Catalog** [20]. Three principal classes have differentiated access:

- **FAA / ICAO regulators** — read-only on Silver and Gold; full audit trail; cannot read operator PII (column-masked).
- **Fleet operators** — read-only on rows scoped to their own operator_id (row-level security).
- **Internal data scientists** — read-only on Silver with hashed aircraft_id.

Bronze is **immutable by policy**. No process — including the data platform team itself — has DELETE permission. This is non-negotiable because Bronze is the legal black-box record. Right-to-be-forgotten requests under GDPR are handled by tombstone records in a separate PII-only Delta table that is joined at query time; Bronze is never modified. Retention is tiered:

- Hot: 60 s (TTL).
- Warm (Pinot): 30 days.
- Cold-Silver: 13 months online, then transitioned to S3 Glacier Instant Retrieval.
- Cold-Bronze: 7 years total, with the final 5 years on Glacier Deep Archive.

This tiering reduces total storage cost by approximately 75 % versus keeping all data on S3 Standard, while remaining compliant with FAA Order 7400.10 retention rules [21].

5. Distributed Processing Strategy

5.1 Hybrid Processing Model

Skylane uses a **hybrid processing model** with a clear bifurcation:

- **Streaming hot path** (Flink, < 50 ms) — collision alerts, geofence breaches, online inference, dashboard updates.
- **Batch cold path** (Spark on Delta, minutes-to-hours) — model training, regulatory reports, cohort analytics, long-horizon trajectory analysis.

The two paths share a single source of truth: the Kafka log is mirrored into Delta Bronze, and both Flink and Spark read from this log (Flink via Kafka, Spark via Delta). This is the **Kappa-flavored Lakehouse** pattern [22], deliberately chosen over the classical Lambda architecture because Lambda's twin-pipeline reconciliation problem is operationally toxic at the scale and criticality of UTM.

5.2 The Spatial Shuffle: The Central Distributed Reasoning Problem

A naïve collision-detection job keys streaming records by `aircraft_id`. Every collision check then requires a cross-partition shuffle — every aircraft must compare against every other aircraft. The cost is $O(n^2)$ network and CPU, which fails at our scale by approximately six orders of magnitude. **The H3 keying strategy** changes this from $O(n^2)$ to $O(n \cdot k)$:

1. At ingestion, every event is enriched with H3(lat, lon, resolution=9). Resolution 9 yields hexagonal cells with edge length ~174 m and area ~0.1 km² [23].
2. Kafka partitions on H3 cell. Flink's `keyBy` is also H3 cell. Therefore every aircraft in a given hex is processed by the same TaskManager, with state co-located.
3. Within each hex, a Flink `KeyedCoProcessFunction` broadcasts the state of the six immediate H3 neighbors via Flink's broadcast state pattern. A 7-cell ring is sufficient because at urban drone speeds (≤ 30 m/s) and a 3-second collision

horizon, the maximum reachable distance is ≤ 90 m — strictly within the immediate hex ring [24].

4. The collision check is therefore $O(k^2)$ per hex per tick, where k is the average number of aircraft per ring (typically 4–8). Aggregated across n aircraft, total work is $O(n \cdot k)$, not $O(n^2)$.

This is the single architectural decision that makes Skylane physically realizable.

5.3 Where Shuffle Still Occurs and How It Is Minimized

Shuffle is not eliminated; it is **localized**. Three remaining shuffle points and their mitigations:

1. **Hex boundary crossings.** When an aircraft crosses an H3 cell boundary, its state must migrate to a new TaskManager. Frequency: at 15 m/s, an aircraft crosses one hex every ~ 12 seconds — 1.5 % of the 10 Hz tick rate. Mitigation: Flink's KeyGroup rebalancing handles state migration transparently; the operational cost is negligible.
2. **Cross-cell trajectory broadcasts.** Each TaskManager receives the state of its 6 neighbors via Flink's broadcast state pattern. This is a controlled broadcast, not a shuffle, with bandwidth proportional to (neighbor cells $\times$ aircraft per cell $\times$ event size) — bounded and predictable.
3. **Periodic rebalancing.** Reactive Flink rebalances task slots roughly every 60 seconds in response to skew. This briefly pauses processing for sub-second windows, which is acceptable because watermark allowance (200 ms) absorbs the gap.

5.4 Skew and Stragglers

Skew is the dominant operational risk. Vertiports, airports, and emergency-response sites concentrate hundreds of aircraft into a single H3 cell. One TaskManager becomes the bottleneck while the rest of the cluster idles. This is the

canonical **straggler** problem from the course's clickstream example, transposed into a spatial domain. Three mitigations operate together:

- **Adaptive sub-partitioning.** A monitoring loop tracks per-cell event rates. Cells exceeding a configurable threshold (e.g., 5,000 events/sec) are dynamically split to H3 resolution 10 (~65 m hexagons), and Flink's parallelism for that key range is increased. This is analogous to **two-phase aggregation** in Spark: a high-cardinality coarse key is decomposed into multiple finer keys.
- **Salting.** The top 1 % hottest hexes (typically major vertiports) are pre-salted: their key is suffixed with a random integer 0–15, distributing their load across 16 TaskManagers. A second-stage merge re-aggregates the salted partitions. This is the standard Spark skew mitigation [25] applied to streaming.
- **Pre-aggregation.** Within a hex, aircraft are summarized into a compact "occupancy vector" (count, mean velocity, density) before being broadcast to neighboring cells. This reduces broadcast bandwidth by a factor of ~50 and prevents broadcast traffic from saturating inter-node links.

5.5 Join Strategies

Skylane uses three distinct join strategies, each chosen for a specific data characteristic:

Join	Left input	Right input	Strategy	Justification
Telemetry × Geofence	1 M events/sec	~10,000 active geofences	Broadcast hash join	Right side fits comfortably in TaskManager heap; eliminates shuffle entirely
Telemetry × Telemetry (collision)	Self-join within hex	Same	Co-partitioned join	Both sides keyed on H3 cell at ingestion; zero shuffle

Join	Left input	Right input	Strategy	Justification
Telemetry × Weather	1 M events/sec	~100 weather stations	Broadcast hash join	Weather table is tiny; updated every 60 s; broadcast variable in Flink
Daily aggregate × Operator dimension	~1 B daily rows	~1,000 operators	Sort-merge / shuffle hash join (Spark, batch)	Cold path; latency unimportant; shuffle acceptable on dedicated nightly cluster

Critically, **no shuffle join occurs on the streaming hot path**. Every real-time join is either broadcast or co-partitioned. This is what makes the 50 ms SLA achievable.

5.6 Reducing Unnecessary Data Movement

Beyond join strategy, three system-wide techniques reduce data movement:

- **Rack-aware Flink placement.** Neighboring H3 cells are placed on the same rack where possible, so broadcast state traffic stays inside the rack switch.
- **Predicate pushdown into Delta.** All Spark/Trino queries push date and H3 predicates into Delta's file-skipping layer. Typical regulatory queries scan < 0.1 % of total data.
- **Event compaction at ingestion.** Aircraft batch up to 5 telemetry samples (50 ms window) before publishing — reducing per-event overhead by 4× while staying well inside the 200 ms watermark allowance.

6. Scalability and Failure Analysis

6.1 Predicted Bottlenecks (in order of failure likelihood)

Bottleneck 1 — Flink RocksDB state size. As traffic concentrates around vertiports during rush hour, per-TaskManager state grows. Each aircraft holds ~30 state vectors over a rolling 3-second window, totaling ~10 KB. At 100,000 aircraft, total state is ~1 GB cluster-wide — manageable. However, during a regional surge (e.g., post-event evacuation), hot cells may concentrate state heavily on a single TaskManager.

Mitigation: **incremental RocksDB checkpoints to S3** (only deltas are uploaded), off-heap memory for state, 5-minute idle TTL on aircraft state, and reactive

rebalancing.**Bottleneck 2 — Kafka partition imbalance.** When an H3 cell concentrates traffic, its corresponding Kafka partition becomes a hot partition. Mitigation: producer-side dynamic re-keying — when a publisher detects that its target hex is saturated, it switches to res-10 keying with a random salt. Consumer Flink jobs are pre-configured to handle both res-9 and res-10 keys.**Bottleneck 3 — Inter-TaskManager broadcast**

bandwidth. Each TaskManager broadcasts neighbor-cell summaries to up to 6 peers. At full scale this approaches 200 MB/s per TM — significant but well within 25 GbE NIC capacity. Mitigation: rack-aware placement, pre-aggregated occupancy vectors, and gossip-style differential updates (only changed fields are broadcast).

Bottleneck 4 — Pinot real-time ingestion. Pinot must keep up with 1 M events/sec for analytical queries. Pinot's published throughput per server is ~150 K events/sec [11].

Mitigation: 8 real-time Pinot servers with replication factor 2 = 16 nodes for

headroom.**Bottleneck 5 — Cross-region replication lag.** Multi-region Kafka MirrorMaker 2 introduces 50–200 ms cross-region lag. Mitigation: collision-critical traffic remains in-region; only cold-path replication is cross-region.

6.2 Failure Modes and Recovery

Failure	Detection	Recovery mechanism	RTO	RPO
Flink Task Manager crash	Heartbeat timeout (10 s)	Restart from last RocksDB checkpoint (every 30 s); re-read Kafka from committed offset	< 15 s	0 (exactly-once)
Flink JobManager crash	ZooKeeper / Kubernetes leader election	Standby JM takes over; jobs continue via HA storage	< 10 s	0
Kafka broker loss	ISR (in-sync replica) protocol	Leader re-election; replication factor 3 ensures continuity	< 5 s	0
Network partition isolates a sector	Heartbeat timeout	Surviving region continues processing its own H3 cells; partition heals via Kafka offset replay	Bounded by partition duration	0
Delta Lake write conflict	Delta optimistic concurrency control	ACID transaction aborts; producer retries	Immediate	0
S3 region unavailability	CloudWatch / S3 health check	Fail over to secondary region; Kafka MirrorMaker 2 replicates state	< 60 s	< 60 s of cold-path data
Cascading cluster overload	Rate-limit alarms	Backpressure propagates to MQTT brokers; aircraft buffer locally for up to 10 s; controlled load shedding	< 30 s recovery	0 if buffered, otherwise

Failure	Detection	Recovery mechanism	RTO	RPO
				per drop policy
ML model corruption	Model drift monitoring	Hot-swap to last known good ONNX checkpoint from MLflow	< 60 s	N/A

The architecture's most elegant failure property comes for free from the H3 keying strategy: **a network partition that isolates a geographic sector does not corrupt the rest of the system**. Each H3 cell's processing is independent. When the partition heals, each side's Kafka offsets are reconciled via standard log replay; no distributed consensus is required across the partition boundary.

6.3 Scaling from 20 TB/day to 200 TB/day

The system is designed for one order-of-magnitude growth without architectural change:

- **Kafka.** Add brokers; increase partitions per H3 cell from 1 to 4. Kafka scales linearly to thousands of brokers in production at LinkedIn and Uber [26].
- **Flink.** Reactive mode auto-scales TaskManagers. The architectural ceiling is JobManager coordination overhead at ~10,000 task slots [27]; beyond that, **federation** is required: split jobs by H3 res-3 region, each with its own JobManager, joined only at the cold path.
- **Delta Lake.** Storage scales linearly on S3. The new bottleneck becomes **compaction** (OPTIMIZE + Z-ORDER on 6 PB), which moves to a dedicated compaction cluster running off-peak.
- **Pinot.** Add servers; partition real-time tables by H3_cell_res4 for routing.

The hard ceiling is the JobManager. Above ~500,000 concurrent aircraft, Skylane must federate into multiple regional clusters with a unified data plane in Delta — analogous to how Google Spanner federates across regions [28].

7. Tradeoff Analysis

Every architectural decision in Skylane was made against a competing alternative. The eight most consequential tradeoffs:

7.1 Exactly-Once vs At-Least-Once Semantics

Skylane uses **exactly-once** end-to-end on the safety-critical path. The cost is approximately 2× throughput overhead from Kafka transactional sinks (two-phase commit) and Flink's barrier alignment [10]. In any other domain, at-least-once with idempotent consumers would be the rational choice. In aviation safety it is not: a duplicated collision alert grounds an entire fleet (operationally catastrophic), and a lost alert kills people (legally catastrophic). The math is not close.

7.2 Streaming vs Batch

Skylane uses **streaming for the hot path, batch for the cold path**. Pure batch fails the 50 ms SLA by three orders of magnitude. Pure streaming fails the cost model: training a deep trajectory model in streaming mode would cost ~5× more than batch and provide no benefit because labels (actual future positions) require time to materialize.

7.3 Spatial Partitioning vs Identity Partitioning

Skylane partitions on **H3 cell, not aircraft_id**. This is the central architectural decision. The cost is occasional state migration on hex boundary crossings. The benefit is collapsing $O(n^2)$ collision detection to $O(n \cdot k)$. The decision is forced; no alternative reaches the SLA.

7.4 Self-Managed Flink/Kafka vs Managed Services

Skylane uses **self-managed Kafka and Flink on Kubernetes** rather than AWS Kinesis + AWS Managed Flink. Cost analysis shows self-management is roughly 40 % cheaper at this scale, primarily due to network-egress pricing. The cost is ~2 dedicated platform engineers. For a system at this scale and criticality, the engineering cost is dominated by the cost savings and the latency control. By contrast, Skylane uses **fully managed S3**

and **Delta Lake** rather than self-hosted HDFS — because S3's eleven-nines durability and operational burden differential are not even close.

7.5 Lakehouse vs Data Warehouse

Skylane uses a **lakehouse (Delta Lake)** rather than a pure warehouse (Snowflake, BigQuery). At 6 PB/year, Snowflake storage cost alone would be ~\$1.5 M/year, vs ~\$150 K on S3 [6][29]. The cost is more engineering effort for governance and query optimization. For a regulated workload requiring open formats and 7-year retention, this is the right tradeoff.

7.6 In-Memory vs Disk-Backed State

Skylane uses **RocksDB-backed Flink state** rather than pure in-memory state. Pure heap state is 2–3× faster but cannot hold the working set during traffic surges. RocksDB with off-heap memory and SSD-backed state files gives ~95 % of the performance with 10× the capacity headroom [30].

7.7 Embedded Inference vs Model Server

Skylane embeds **ONNX inference inside Flink TaskManagers** rather than calling out to a TensorFlow Serving or KServe model server. The cost is tighter coupling between model lifecycle and pipeline lifecycle. The benefit is eliminating a 5–10 ms RPC round-trip — which alone consumes 10–20 % of the entire 50 ms latency budget.

7.8 Active-Active vs Active-Passive Multi-Region

Skylane is **active-active across two regions**, doubling the infrastructure cost. The alternative — active-passive with a 1–2 minute failover — was rejected because aviation safety mandates zero RPO. A 60-second blind window over a major metropolitan airspace is not survivable.

8. Analytics, Dashboarding, and ML Integration

8.1 Analytics and Dashboards

Three classes of consumers extract value from the system:

- **Real-time operational consumers** — fleet operators and city dispatchers receive collision alerts, geofence breaches, and aircraft-state updates over WebSocket connections that subscribe directly to Kafka topics. Latency: end-to-end ≤ 50 ms.
- **Sub-second analytical consumers** — operator dashboards (Grafana, Apache Superset) query Apache Pinot for metrics such as "active aircraft per sector", "average sector congestion over last 5 minutes", and "anomaly rate by operator". Latency: < 1 sec per query.
- **Ad-hoc and regulatory consumers** — FAA investigators, NTSB accident reconstructors, and internal data scientists query Delta via Trino with sub-minute latency for arbitrary historical questions over multi-petabyte history.

8.2 Machine Learning Pipelines

Three production ML models, each integrated at a distinct point in the pipeline:

Model 1 — Trajectory Prediction (LSTM). Predicts aircraft position 5 seconds into the future given the last 3 seconds of state. Trained nightly on Delta Silver. Inferred online at 10 Hz per aircraft inside Flink via ONNX Runtime. Accuracy is critical because collision detection augments simple kinematic extrapolation with this learned model for non-linear maneuvers.

Model 2 — Power-Draw Anomaly Detector (Isolation Forest). Flags aircraft whose measured power draw deviates from expected given mass, wind, and maneuver. Used to infer unauthorized payloads, motor degradation, and spoofed flight plans. Trained nightly, inferred online inside Flink. Streaming inference allows ground-truth-free detection within seconds of takeoff.

Model 3 — Geofence Violation Risk (Gradient Boosted Trees). Predicts the probability of an aircraft entering a restricted zone in the next 60 seconds. Drives proactive dispatcher re-routing. Trained weekly, inferred online.

8.3 Feature Pipeline

Features flow through a single Feast feature store with split backends [14]:

- **Online features** — current aircraft state, current weather, current sector congestion. Served from Redis with < 1 ms latency. Used by streaming inference.
- **Offline features** — historical aircraft behavior, operator history, seasonal patterns. Served from Delta Silver. Used during training and for batch backfills.

This split eliminates the **training-serving skew** problem that breaks most production ML systems [31]: both training and serving read from the same logical feature definitions.

8.4 Data Freshness Requirements

Use case	Freshness budget
Online inference (streaming)	< 100 ms
Operator dashboards	< 1 sec
Daily operational reports	< 6 hours
Regulatory reports	< 24 hours
Model retraining	nightly (24 hours)

The architecture meets each budget by tier: hot tier serves the < 100 ms case, warm tier the < 1 sec case, cold tier everything else.

9. Conclusion

9.1 Summary of Design

Skylane is a stateful spatial streaming architecture for next-generation Unmanned Aircraft System Traffic Management. It ingests one million telemetry events per second from 100,000 concurrent autonomous aircraft, generates collision alerts in under 50 milliseconds end-to-end, and persists 15–20 TB of flight data per day in a 7-year regulatory record. The architecture comprises five layers — edge ingestion (MQTT to Kafka), stateful stream processing (Flink), tiered storage (Redis → Pinot → Delta Lake

on S3), serving (WebSocket + Pinot + Trino), and machine learning (Spark training, ONNX online inference). The central design insight is that **spatial proximity, not device identity, must drive partitioning**: by keying ingestion and processing on H3 hexagonal cells, the system reduces an otherwise intractable $O(n^2)$ collision-detection workload to a parallelizable $O(n \cdot k)$ workload.

9.2 Strengths

- **Spatial-first partitioning** eliminates the dominant shuffle cost and is the architectural feature most likely to be of independent research interest.
- **Hybrid Kappa-Lakehouse** provides a single source of truth across three latency tiers without the reconciliation pathology of Lambda.
- **Exactly-once semantics** are defensible against aviation safety standards rather than treated as optional.
- **Open formats throughout** (Parquet, Delta, ONNX, MQTT, Kafka protocol) prevent regulatory vendor lock-in.
- **Failure isolation by geography** is a free byproduct of H3 keying: a partitioned region degrades cleanly without corrupting the rest of the system.

9.3 Limitations

- **Centralized control plane.** Flink's JobManager is a coordination ceiling near ~10,000 task slots. True continent-scale UTM requires federation, which is described in Section 6.3 but not architected here.
- **Multi-region cost.** Active-active operation doubles infrastructure cost; this is unavoidable given aviation safety RPO requirements but is a real economic constraint.
- **2D spatial indexing.** H3 is a 2D hierarchical spatial index. Full 4D indexing (latitude, longitude, altitude, time) is an open research problem; current handling of altitude is via a separate dimensional bucket layered on top of H3, which is workable but inelegant.

- **Edge dependence.** The architecture assumes reliable 5G uplink. In practice, edge buffering, store-and-forward at the aircraft, and tolerance for late-arriving telemetry are required complements.
- **Cold-start ML.** New aircraft types arrive without training data; the trajectory model must fall back to kinematic extrapolation until sufficient data accumulates.

9.4 Future Work

Three research directions naturally extend this design:

1. **True 4D spatiotemporal indexing**, possibly via S2-derived 3D hierarchies or learned spatial indexes [32].
2. **Federated streaming** across regional Flink clusters with a unified Delta plane, enabling continental UTM.
3. **Reinforcement learning for dynamic airspace allocation**, replacing heuristic geofence rules with learned policies over the same data substrate.

9.5 Final Reflection

The deepest lesson of this design is that the physics of distributed systems and the physics of airspace are the same problem viewed twice: locality wins. When data is partitioned along the same axis as the underlying physical reality , geographic position, in this case , the rest of the architecture composes naturally. Every component in Skylane exists because a specific quantitative requirement forced it into existence, not because it was fashionable. That is what separates a Big Data design from an inventory of Big Data tools.

References

- [1] Federal Aviation Administration, *Unmanned Aircraft System Traffic Management (UTM) Concept of Operations, Version 2.0*, Washington, DC: FAA, 2020.[2] National Aeronautics and Space Administration, *Urban Air Mobility (UAM) Vision Concept of*

Operations (ConOps), UAM Maturity Level 4, NASA Tech. Rep. NASA/CR-20210018320, 2021.[3] PostgreSQL Global Development Group, "pgbench benchmarking results, PostgreSQL 16 on AWS r6i.32xlarge," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/pgbench.html>[4] N. Marz and J. Warren, *Big Data: Principles and best practices of scalable real-time data systems*. Shelter Island, NY: Manning, 2015.[5] McKinsey & Company, "Up in the air: How do consumers view advanced air mobility?" McKinsey Center for Future Mobility, 2023.[6] Amazon Web Services, "Amazon S3 Service Level Agreement," 2024. [Online]. Available: <https://aws.amazon.com/s3/sla/>[7] OASIS, *MQTT Version 5.0 OASIS Standard*, 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>[8] J. Kreps, N. Narkhede, and J. Rao, "Kafka: a distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.[9] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache Flink: Stream and batch processing in a single engine," *IEEE Data Engineering Bulletin*, vol. 38, no. 4, pp. 28–38, 2015.[10] P. Carbone, S. Ewen, G. Fora, S. Haridi, S. Richter, and K. Tzoumas, "State management in Apache Flink: Consistent stateful distributed stream processing," *Proc. VLDB Endowment*, vol. 10, no. 12, pp. 1718–1729, 2017.[11] J. F. Im, K. Gopalakrishna, S. Subramaniam, M. Shrivastava, A. Tumanov, and S. Jaiswal, "Pinot: Realtime OLAP for 530 million users," in *Proc. ACM SIGMOD*, 2018, pp. 583–594.[12] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Zhu, M. Murthy, J. Torres, H. van Hovell, A. Ionescu, A. Łuszczak, M. Świtakowski, M. Szafranski, X. Li, T. Ueshin, M. Mokhtar, P. Boncz, A. Ghodsi, S. Paranjpye, P. Senster, R. Xin, and M. Zaharia, "Delta Lake: High-performance ACID table storage over cloud object stores," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, 2020.[13] R. Sethi et al., "Presto: SQL on everything," in *Proc. IEEE ICDE*, 2019, pp. 1802–1813.[14] Tecton and the Linux Foundation, "Feast: Feature store for machine learning," 2024. [Online]. Available: <https://feast.dev>[15] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," in *Proc. CIDR*, 2021.[16] Databricks, "What is the medallion lakehouse architecture?" 2023. [Online]. Available: <https://www.databricks.com/glossary/medallion-architecture>[17] Apache Software Foundation, "Apache Parquet documentation," 2024. [Online]. Available: <https://parquet.apache.org/docs/>[18] Y. Collet and M. Kucherawy, *Zstandard Compression and the application/zstd Media Type*, IETF RFC 8478, 2018.[19] Z-

ordering for multi-dimensional data clustering, in M. Armbrust et al., "Delta Lake OPTIMIZE and Z-ORDER," Databricks documentation, 2024.[20] Databricks, "Unity Catalog: Unified governance for all data and AI assets," 2024. [Online]. Available: <https://www.databricks.com/product/unity-catalog>[21] Federal Aviation Administration, *Order JO 7400.10G, Special Use Airspace*, 2024.[22] J. Kreps, "Questioning the Lambda architecture," *O'Reilly Radar*, July 2014.[23] Uber Technologies, "H3: Hexagonal hierarchical geospatial indexing system," 2018. [Online]. Available: <https://h3geo.org>[24] I. Brodsky, "H3: Uber's hexagonal hierarchical spatial index," Uber Engineering Blog, 2018.[25] H. Karau and R. Warren, *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. Sebastopol, CA: O'Reilly Media, 2017, ch. 6.[26] T. Rao, "How LinkedIn customizes Apache Kafka for 7 trillion messages per day," LinkedIn Engineering Blog, 2019.[27] Apache Flink Documentation, "Flink reactive mode and scaling limits," v1.18, 2024. [Online]. Available: <https://nightlies.apache.org/flink/>[28] J. C. Corbett et al., "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems*, vol. 31, no. 3, art. 8, 2013.[29] Snowflake Inc., "Snowflake Storage Pricing Guide," 2024. [Online]. Available: <https://www.snowflake.com/pricing/>[30] S. Dong, M. Callaghan, L. Galanis, D. Borthakur, T. Savor, and M. Stumm, "Optimizing space amplification in RocksDB," in *Proc. CIDR*, 2017.[31] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015, pp. 2503–2511.[32] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proc. ACM SIGMOD*, 2018, pp. 489–504.